University of Piraeus
School of Information and Telecommunications Technologies
Department of Digital Systems

Level: Postgraduate Program of Study

Network Security

Case 2

Supervisor: Prof. Christos Xenakis

Kalaitzidis Ioannis ioannis_kalaitzidis@ssl-unipi.gr

Piraeus
01/11/2025

Abstract

Packet sniffing at a low level using raw sockets on Kali Linux (VMware), with the network card in promiscuous mode. Packets are collected in a pcap file and graphically analyzed with Wireshark.

Contents

Introduction

In this exercise we deal with packet sniffing at a low level using raw sockets. A socket is generally a communication mechanism that the operating system uses for applications to exchange data over a network. Raw sockets are a special type of socket that allows the application to directly access network packets, without the normal processing of the protocol by the kernel. In other words, raw sockets bypass standard kernel filtering and interfere with the process of sending packets to applications. Thus, in the first phase the packets are not filtered or separated for delivery to UDP/TCP sockets, and we can see all the bytes and headers as received by the network card. In this way we gain access to information that we would not otherwise see easily.

A network card (NIC) is the physical device that connects the computer to the network, and each NIC has a unique MAC address. The NIC works in different modes: promiscuous mode (in wired and wireless networks) where it reads all frames passing through the interface (i.e. the network card to the operating system), and monitor mode (in wireless networks only) where it reads frames on the specific wireless channel (i.e. at the frequency that the network card transmits/receives). For the exercise, we will activate the promiscuous mode, so that the raw socket receives all the traffic of the interface and we can analyze packets.

The VM sees a virtual NIC managed by the host. This virtual card is not the same as the real physical Wi-Fi card. So, even if the host is connected wirelessly, the VM does not have direct access to Wi-Fi traffic and only sees what the host provides to it depending on the mode (NAT or Bridged).

- NAT: The VM shares the network output with the host and gets an internal IP. External traffic is displayed with the host's IP. Practically, the VM mainly sees its own traffic, which limits the ability to sniffing the traffic of other devices in the LAN.
- Bridged: The VM behaves like a separate device on the local network (it takes IP from the router) and can exchange packets directly with other devices on the LAN. This allows for wider sniffing, but exposes the VM to the local network and carries greater risk and legal liabilities.

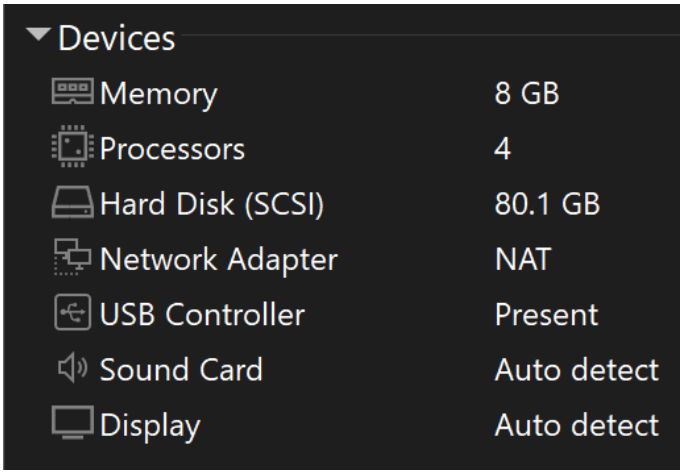
Practical piece of work

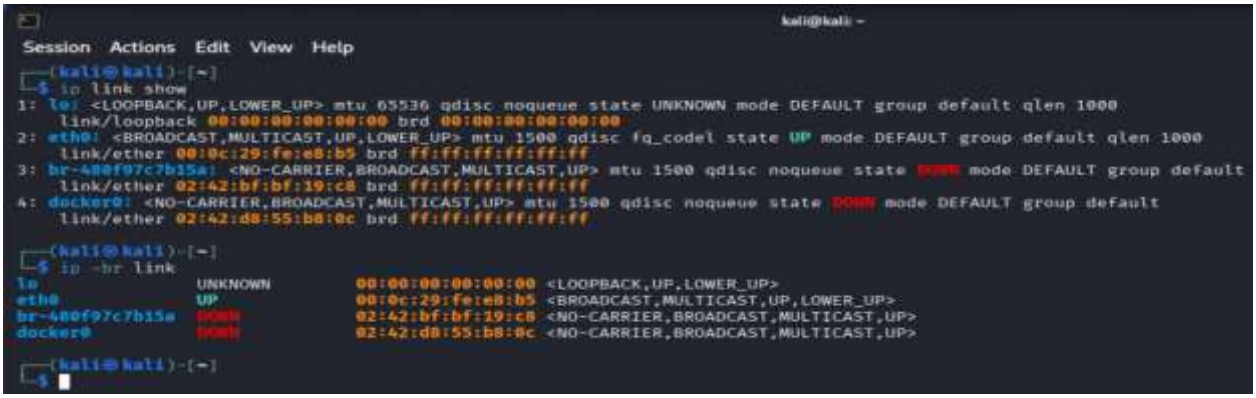
Introduction:

We will run the exercise on Kali Linux (kali-linux-2025.3-vmware-amd64) through VMware Workstation Pro (latest version) because we need root privileges to work the raw sockets and to change the card mode (promiscuous). For isolation and security reasons, we choose NAT as the default. With this setting, the VM is not directly exposed to the local network and we mainly see the traffic that the VM itself generates/receives, without accidentally collecting traffic from other LAN devices. If we need motion analysis of other devices, then we will choose Bridged or use a physical Wi-Fi adapter in monitor mode. In the VM we will also run Wireshark in parallel for graphical analysis and filter application.

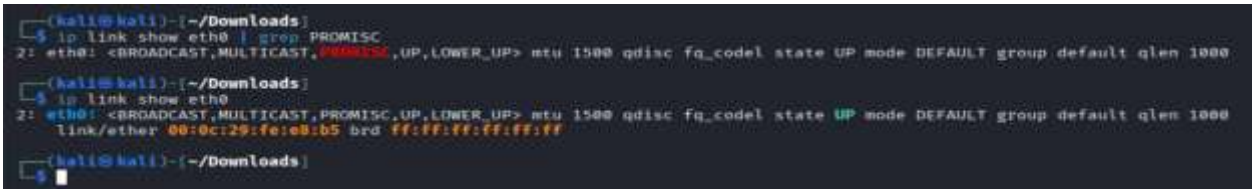
Initiating Procedures:

From the characteristics of the resources we have available for this OS, we notice that the network adapter is in NAT. So we will not make any modification to this setting.

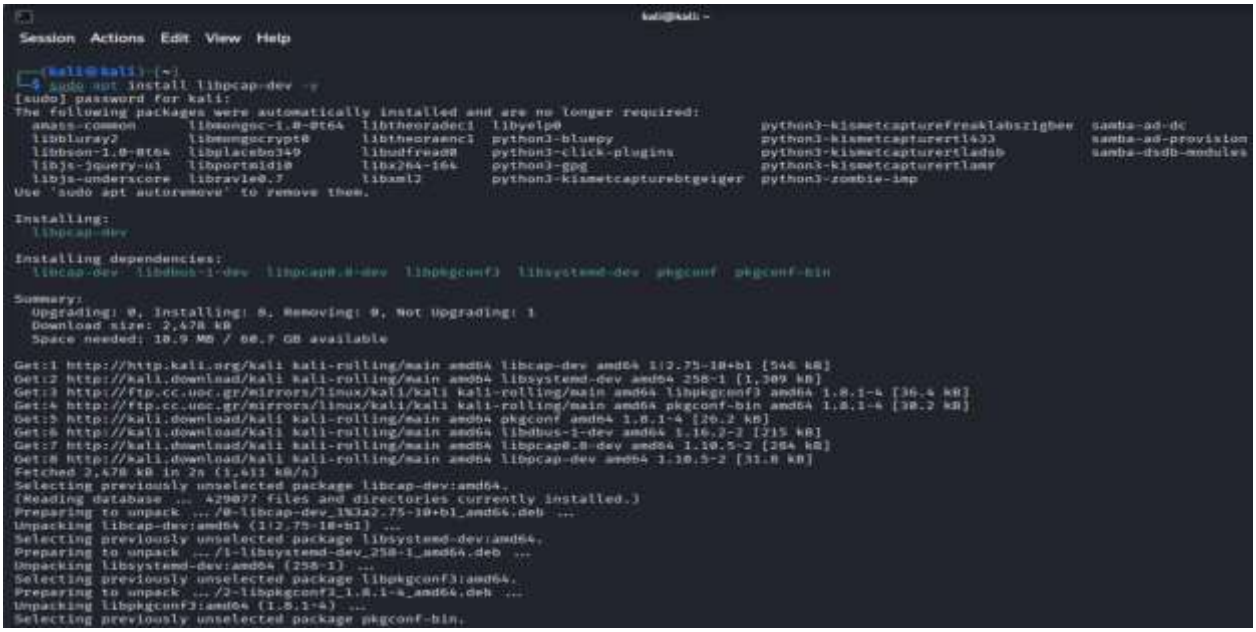




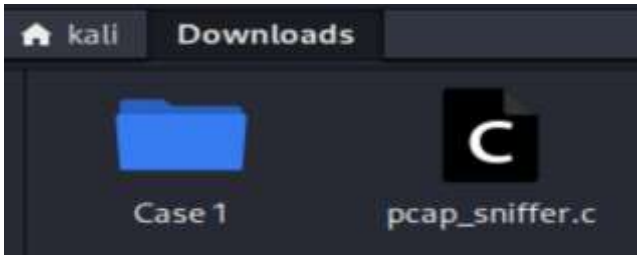
We notice that either with the **ip link show** command or with the ip **-br link** command we control our interface. The interface we will use in the exercise is eth0 because it is the only one that is active (UP) and corresponds to the virtual card given to us by VMware (via NAT).



The next step is to check if the promiscuous mode is active. With the command **ip link show eth0 | grep PROMISC** (the same is done with the command **ip link show eth0**) we can check if the specific mode is active. If it is not, we do it with the command **sudo ip link set eth0 promisc on** (and respectively to disable it it is the same command but ending **in off** instead of **on**).



After completing the above procedures, we will download the libpcap library. Then we will make packet sniffer in the programming language in C via the library. We made the file in pcap_sniffer.c in the Downloads folder and it includes the modified code based on that of the slide provided by the instructor.



We go to the Downloads folder where we have the file and we will compile it.

```
┌──(kali㉿kali)-[~]
└─$ cd ~/Downloads && ls -l *.c
-rw-rw-r-- 1 kali kali 1619 Oct 31 10:07 pcap_sniffer.c
```

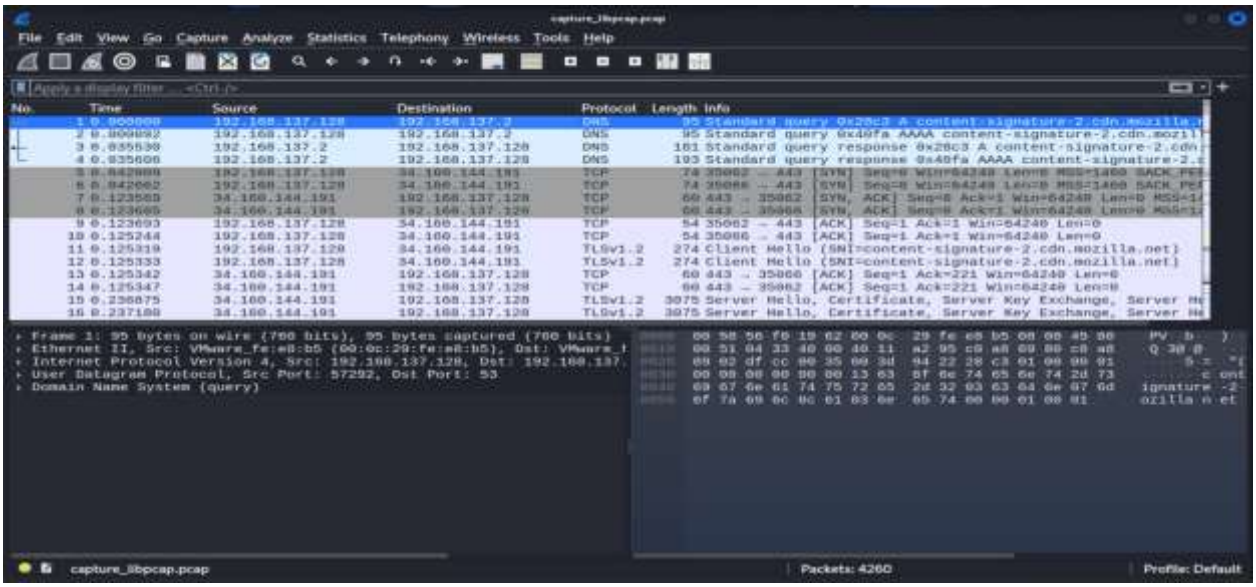
With this command, in addition to "entering" the Downloads folder, we also searched for any file ending in .c. The only file in this case was the sniffer so we will proceed to compile it.

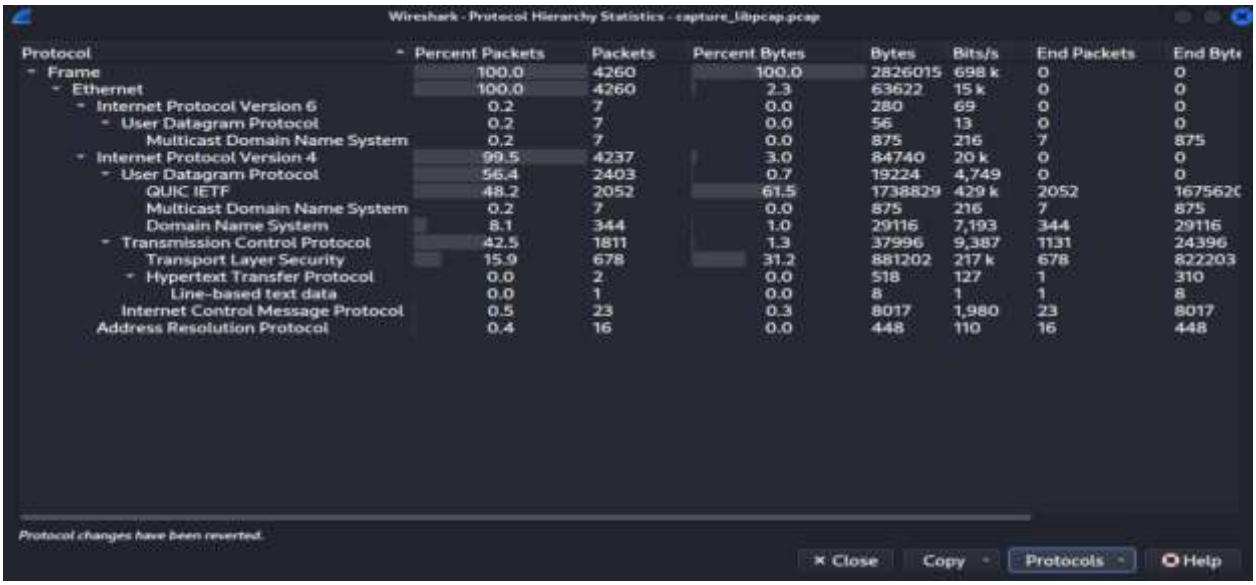
```
┌──(kali㉿kali)-[~/Downloads]
└─$ gcc pcap_sniffer.c -o pcap_sniffer -lpcap

┌──(kali㉿kali)-[~/Downloads]
└─$ sudo ./pcap_sniffer
Usage: ./pcap_sniffer <interface>

┌──(kali㉿kali)-[~/Downloads]
└─$ sudo ./pcap_sniffer eth0
Capturing packets on eth0... Press Ctrl+C to stop.
Captured packet (42 bytes)
Captured packet (60 bytes)
Captured packet (71 bytes)
Captured packet (70 bytes)
Captured packet (122 bytes)
Captured packet (74 bytes)
Captured packet (376 bytes)
Captured packet (74 bytes)
Captured packet (74 bytes)
Captured packet (115 bytes)
Captured packet (66 bytes)
Captured packet (79 bytes)
Captured packet (1294 bytes)
Captured packet (933 bytes)
Captured packet (74 bytes)
Captured packet (74 bytes)
Captured packet (1034 bytes)
Captured packet (66 bytes)
Captured packet (80 bytes)
Captured packet (70 bytes)
^C
Stopping capture and closing files...

┌──(kali㉿kali)-[~/Downloads]
└─$
```

After compiling the sniffer with the command **gcc pcap_sniffer.c -o pcap_sniffer -lpcap** through the library and making sure that it will not have any problems during compilation, we run the sniffer file and put the port that is open as the interface (**sudo ./pcap_sniffer eth0**). This is how the recording of the packets starts and we can stop the process by pressing the ctrl + C buttons on the keyboard. Completing the above process, The file (**capture_libpcap.pcap**) that we will open via Wireshark is created and includes all the information about the packages. We will open the file with the wireshark command <name of the file> (**wireshark capture_libpcap.pcap).**

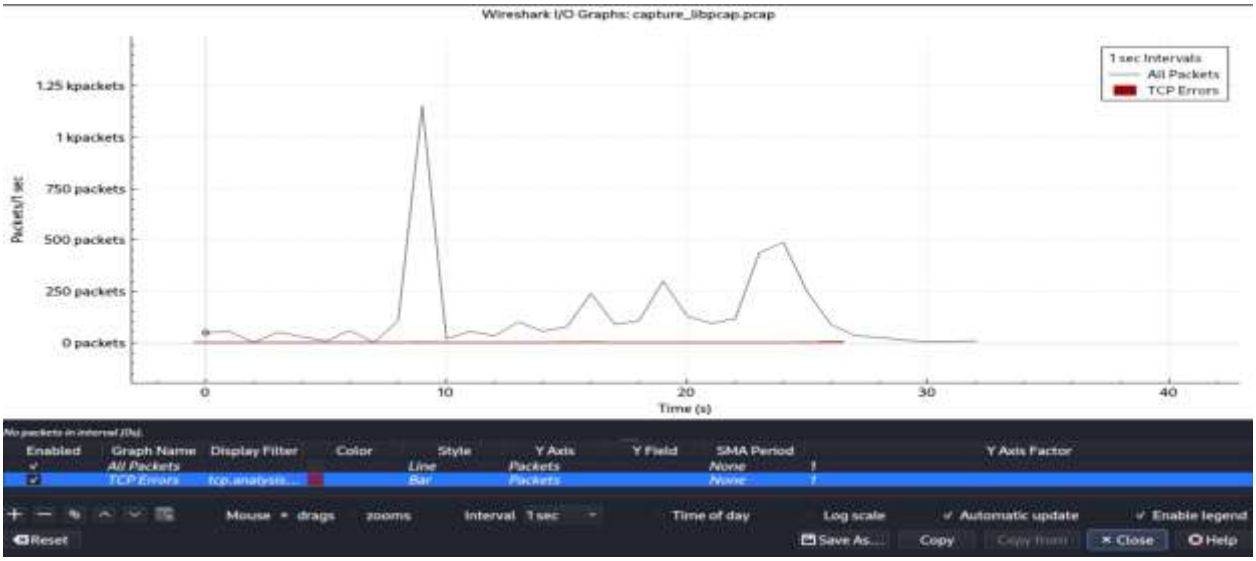
wireshark will open and display all the packets.

Then we can select the Statistics to draw the first conclusions:





- From the analysis of the hierarchy of protocols, it can be observed that the majority of traffic is based on IPv4, with UDP (56.4%) and TCP (42.5%) being the dominant transport protocols.



- From the flow graph (I/O Graph) there is a change in the number of packets per second, with a sharp peak at about 10 seconds and smaller increases thereafter. This increase indicates periods of intense network activity, possibly when loading data or initiating new connections. No significant TCP errors are detected, indicating smooth operation and stable data transfer.

Conclusion:

The exercise demonstrated the practical operation of a packet sniffer at a low level and the effectiveness of Wireshark in analyzing and visualizing network traffic.

Bibliography

Moon, S. (2020, July 26). How to code raw sockets in C on Linux - BinaryTides. Retrieved from https://www.binarytides.com/raw-sockets-c-code-linux/

nospaceships/raw-socket-sniffer: Packet capture on Windows without a kernel driver. (n.d.). Retrieved from https://github.com/nospaceships/raw-socket-sniffer

packet(7) - Linux manual page. (n.d.). Retrieved from https://man7.org/linux/man-pages/man7/packet.7.html

Harrichael. (2014, July 25). C raw socket receive or sniff incoming packets only. Retrieved from https://stackoverflow.com/questions/24964586/c-raw-socket-receive-or-sniff-incoming-packets-only

Raw Socket programming - Efficient Packet Sniffer - Applications / Programming - Unix Linux Community. (2013, June 26). Retrieved from https://community.unix.com/t/raw-socket-programming-efficient-packet-sniffer/331041/10

Ssaf. (n.d.). How raw sockets work? Retrieved from https://superuser.com/questions/1678667/how-raw-sockets-work

GeeksforGeeks. (2025, July 15). Kali Linux sniffing and spoofing. Retrieved from https://www.geeksforgeeks.org/linux-unix/kali-linux-sniffing-and-spoofing/

ChatGPT. (n.d.). Retrieved from https://chatgpt.com/